1 1 4 1 4 0

Action-Prefix Recognition Instruction (§ . .) — LAYER- carrier block

Field	Value
Revision	6
Created	2026-06-09
Last modified	2026-07-16T11:00:00Z
Status	active
Scope	the reusable verbatim instruction block embedded into every agent context carrier
Authority	DESIGN.md §5 + RULE_DRAFT.md Part B (this directory)

> **Revision 6 (2026-07-16):** added the fourth **§11.4.213 reporting-family directive** `FEATURE` — a RESEARCH-SCHEDULING directive (all six §11.4.140 forms incl. the single-colon `FEATURE:`) that SCHEDULES rather than synchronously executes a deep, enterprise-grade research + implementation- planning effort, by invoking the SAME §11.4.202 `report_item.sh` engine (never a duplicate implementation) plus a new durable `docs/requests/feature_queue.md` queue.
>
> **Revision 5 (2026-07-16):** added the three **severity / handling-priority markers** `CRITICAL` / `IMPORTANT` / `NOTE` (registry rows, all six §11.4.140 forms incl. the single-colon `CRITICAL:` / `IMPORTANT:` / `NOTE:`). Registering `NOTE` PROMOTES `NOTE:` from an ordinary-prose NO-OP to a registered action — the ordinary-prose single-colon examples are now `TODO:` / `WARNING:` / `FIXME:`.
>
> **Revision 4 (2026-07-15):** added the **§11.4.202 reporting directives** (`ISSUE` / `BUG` / `TASK`) + the **single-colon `NAME:` form** (form 6), registered-action-only so ordinary prose (`TODO:` / `FIXME:`) never expands and never asks. A report now auto-creates a fully-populated, fully-synced workable item; a prose-only acknowledgement is a §11.4 bluff (§11.4.197).
>
> **Revision 3 (2026-07-14):** added the **sub-system shortcut** extension — a grammar-shaped token that names an incorporated sub-system / submodule (from the registry `subsystems:` catalogue OR recursive `.gitmodules` discovery) expands to a sub-system context injection. No new grammar; the same five forms.
>
> **Revision 2 (2026-07-02):** added the fifth grammar form — the arrow form `ACTION_NAME ---> <rest>` (and namespaced `PREFIX::ACTION_NAME ---> <rest>`) — as a first-class equivalent delimiter alongside `::` and `/` , plus the second registered action `REMINDER` (verify-don't-assume status re-surfacing).

This file holds the **single canonical LAYER-1 recognition-instruction block**. The conductor (or the §11.4.35 propagation machinery) embeds the block between the two markers below — VERBATIM, identical text — into every agent context carrier the constitution maintains (`CLAUDE.md` , `AGENTS.md` , `QWEN.md` , `GEMINI.md`).

That textual identity across all four carriers IS the LAYER-1 universality: every CLI agent reads the same instruction via its own carrier and applies the prefix expansion itself (per §11.4.35), so the system works on every agent with zero extra setup.

The block is self-contained: even an agent with NO registry-file access still has the most-used `BACKGROUND` expansion inline, while the registry (`actions/registry.yaml`) remains the extensible source of truth for every other action.

> NOTE: this is the canonical block text. It is NOT applied to the live governance files (`CLAUDE.md` / `AGENTS.md` / `QWEN.md` / `GEMINI.md`) by this machinery — those are conductor-owned (§11.4.58 contention paths). The operator lands it via the §11.4.26 constitution-update pipeline.

§11.4.140 — Universal action-prefix system (`ACTION_NAME ::`) (User mandate, 2026-06-09; GRAMMAR_ADDENDUM 2026-06-09; arrow form 2026-07-02). When a user prompt's FIRST non-blank line starts with a recognised action prefix, you MUST: (1) look the action token up in the action registry `constitution/actions/registry.yaml` (or `$HELIX_ACTION_REGISTRY`); (2) if it is a registered action, REPLACE the prefix with that action's `expansion` text and apply its `rules` ; (3) execute the remainder of the prompt under the expanded instruction. **Five EQUIVALENT forms** — same action, same expansion, same execution: (1) `ACTION_NAME :: <rest>` (bare `::`), (2) `PREFIX::ACTION_NAME :: <rest>` (namespaced `::`), (3) `/ACTION_NAME <rest>` (bare slash), (4) `/PREFIX::ACTION_NAME <rest>` (namespaced slash), (5) `ACTION_NAME ---> <rest>` (bare arrow) ≡ `PREFIX::ACTION_NAME ---> <rest>` (namespaced arrow). Thus `BACKGROUND :: x` ≡ `DEFAULT::BACKGROUND :: x` ≡ `/BACKGROUND x` ≡ `/DEFAULT::BACKGROUND x` ≡ `BACKGROUND ---> x` ≡ `DEFAULT::BACKGROUND ---> x` . `PREFIX` is an action NAMESPACE; the reserved default namespace is **`DEFAULT`** , and an action runs WITH or WITHOUT the prefix. Grammar (all hold): anchored at the FIRST non-blank line only (mid-prose tokens never match); the action token AND the namespace are UPPERCASE-only `[A-Z][A-Z0-9_]*` (lowercase never matches); the namespace separator `::` inside the token carries NO surrounding spaces (`PREFIX::ACTION_NAME`), DISTINCT from the action-body separator `" :: "` (one ASCII space on each side of `::` — avoids C++ `Foo::Bar` , YAML `key: value` , URLs) in forms 1/2, the arrow-body separator `" ---> "` (one ASCII space on each side) in form 5, and the slash-body separator (one space) in forms 3/4; stacked prefixes (`A :: B :: rest`) apply outer-to-inner, left-to-right (expand `A` , re-scan, expand `B` , then the residual is the task); a leading `\` escapes the prefix for the `::` , arrow, and slash forms (`\BACKGROUND :: x` , `\BACKGROUND ---> x` , `\/BACKGROUND x` — treat literally, strip the backslash, NO expansion) so action names can be discussed. **Conflict rule (slash form):** `/ACTION_NAME` (form 3) is honored as the action ONLY when `ACTION_NAME` (case-folded) does not collide with a built-in/host slash command (registry `slash_bare: auto` + `slash_conflicts: [..]`); form 4 (`/PREFIX::ACTION_NAME`) is ALWAYS unambiguous and always honored. An unknown token that matches the grammar shape (any of the 5 forms) but is NOT registered is NEVER silently

expanded or silently dropped — ask which registered action was meant (§11.4.66 / §11.4.105) or treat it as a literal prompt, NEVER invent an expansion (§11.4.6); any prompt not satisfying the grammar is an ordinary prompt and the system is a no-op. The registered action **`BACKGROUND`** expands to: *"The following prompt that we will provide MUST BE executed in background in parallel with all main work streams using the subagents-driven development approach! All work done MUST PRODUCE rock solid evidence covered with hard physical proof(s) that all done is working as expected and as specified without any false results and without any bluff!"* (composes §11.4.20 / §11.4.70 subagent-driven, §11.4.58 / §11.4.103 parallel streams, §11.4.89 background execution, §11.4.5 / §11.4.69 / §11.4.107 captured physical evidence, §11.4 anti-bluff). A second registered action **`REMINDER`** re-surfaces previously-scheduled, CRITICAL, status-UNCERTAIN work: FIRST verify the ACTUAL current status from captured evidence (never assume done or not-done — §11.4.6 no-guessing), THEN act on the delta — report the captured proof if genuinely complete, resume from the exact point if partial, action it NOW if not started, or surface the block per §11.4.66/ §11.4.101 — always producing a status verdict (composes §11.4.6 / §11.4.87 / §11.4.94 / §11.4.97 / §11.4.103 / §11.4.108 / §11.4.130 / §11.4.147).

Severity / handling-priority markers (§11.4.140 extension, 2026-07-16). Three further registered actions tag the REMAINDER of the prompt with a HANDLING PRIORITY (as `BACKGROUND` tags it "run in background"): **`CRITICAL`** (highest-priority / potentially release-blocking — address with maximum urgency

- rigor ahead of lower-priority work, track it, auto-activate §11.4.102 systematic-debugging when an issue is involved, do NOT defer it), **`IMPORTANT`** (high-priority, above routine work but below CRITICAL — track it, full anti-bluff rigor), and **`NOTE`** (capture the remainder as durable CONTEXT — in the operator request-history ledger §11.4.208 / §11.4.210 + persistent agent memory when it is a non-obvious project fact — applied when relevant, NOT an urgent action unless it contains an explicit action; §11.4.6 record only what the note says, never invent). All six grammar forms work (`CRITICAL: x` ≡ `CRITICAL :: x` ≡ `CRITICAL ---> x` ≡ `/CRITICAL x` ≡ `/DEFAULT::CRITICAL x`).

> These markers change HOW the remainder is handled (priority + rigor); they do NOT pick a workable-item Type — when the remainder is a report the §11.4.202 BUG / TASK / ISSUE machinery still classifies + tracks it.

Sub-system shortcuts (§11.4.140 extension, 2026-07-14). When the first-non-blank-line grammar-shaped token is NOT a registered action but NAMES an incorporated SUB-SYSTEM / submodule, it expands to a SUB-SYSTEM CONTEXT injection (repository + org + where-checked-out + §11.4.28 equal-codebase / decoupling / dependency-layout + §11.4.37 fetch-first + §11.4.113 no-force-push + §11.4.183 full-constitution + §11.4/§11.4.69 anti-bluff) and the remainder of the prompt is the task to perform ON that sub-system — e.g. `HELIXQA :: run the atmosphere bank`, `PRESENTER ---> fix the pill`, `/CONTAINERS rebuild the pod`. Two data sources feed resolution: (a) the curated Helix-ecosystem catalogue in `subsystems:` of the registry (human abbreviations like `HXOTA` → HelixOTA), and (b) RECURSIVE `.gitmodules` discovery from the invoking project's own root, so EVERY submodule anywhere in the graph — and any newly-added one — auto-derives its UPPERCASE alias tokens (name / snake / camel-split / abbreviation) out-of-box with no hand-maintained list. A registered behavioral action ALWAYS wins a token collision; duplicate checkouts of the same submodule collapse to one sub-system; an ambiguous (multi-target) or lowercase token NEVER expands and falls to the clarify path (§11.4.6). Same five grammar forms, same first-non-blank-line anchor, same `\`-escape. The distinguishing result field is `kind` ∈ { `action`, `subsystem` }.

Reporting directives + the single-colon form (§11.4.202, 2026-07-15). Three further registered actions turn a plain-language report into a REAL, tracked, fully-synced workable item — never a prose acknowledgement: **`BUG`** (Type=Bug), **`TASK`** (Type=Task), and **`ISSUE`** (a generic report, CLASSIFIED into the §11.4.16 CLOSED set {Bug | Feature | Task} — `ISSUE` is a reporting CHANNEL, never a fourth type). They add a SIXTH equivalent grammar form — the **single-colon** **`NAME: <rest>`** form operators actually type (`ISSUE: subtitles are late`) — alongside the five above (`BUG :: x` ≡ `BUG ---> x` ≡ `/DEFAULT::BUG x` ≡ `BUG: x`). The single-colon form is **REGISTERED-ACTION-ONLY by construction**: a single-colon token that is NOT a registered action is a NO-OP (an ordinary prompt), NEVER an ASK and NEVER a sub-system shortcut — because ordinary prose (`TODO:` ,

`WARNING:` , `FIXME:`) shares that shape and must never be questioned (§11.4.6); the other five forms keep their ASK-on-unknown behaviour. Host collisions use the EXISTING conflict mechanism: the bare `/bug` is a documented Claude Code built-in, so `BUG` declares `slash_conflicts: [bug]` and its bare slash form is not honored — `/DEFAULT::BUG …` is always unambiguous, and `BUG: …` / `BUG :: …` / `BUG ---> …` are never affected. On any of these directives you MUST create the item (Status §11.4.15 + Type §11.4.16 + stable id §11.4.54 + comprehensive structured description §11.4.148 / §11.4.171) via the project's reporting engine (`constitution/scripts/reporting/report_item.sh`), which drives the FULL sync — DB single-source-of-truth (§11.4.93 / §11.4.95) → every derived document regenerated FROM the DB (§11.4.12 / §11.4.53 / §11.4.65 / §11.4.106) → every configured external tracker (§11.4.148 D5) — SKIPPING with an honest reason any tracker whose credentials or client are absent (§11.4.10 / §11.4.6 / §11.4.3) and NEVER faking a push. A report that is discussed but never tracked is a §11.4 PASS-bluff at the requirements-intake layer (§11.4.197).

FEATURE — research-scheduling directive (§11.4.213, 2026-07-16). A fourth reporting-family action, structurally parallel to BUG/TASK/ISSUE but distinct in kind: FEATURE SCHEDULES rather than synchronously executes. The remainder of the prompt describes a feature to research and plan; the directive creates a Type=Task / Status=Queued workable item NOW by invoking the SAME `report_item.sh` engine (never a duplicate implementation of the item-creation / DB-sync / tracker-push machinery) and enqueues it in a durable `docs/requests/feature_queue.md` queue so it can never be silently dropped. The item's comprehensive description embeds the full research mandate as its acceptance-defining work program — deep web research on best incorporation (§11.4.8 / §11.4.99 / §11.4.150), systematic-debug of every enumerated weak-spot / gap / danger-zone with a risk-free rock-solid design per gap (§11.4.102), always enterprise-grade / bleeding-edge / innovative, exhaustive documentation down to lines-of-code + micro-POCs + diagrams + SQL

- templates (§11.4.65 / §11.4.73), reuse-first investigation of vasic-digital / HelixDevelopment components kept decoupled (§11.4.28 / §11.4.74 / §11.4.177), full test-type + Challenges + HelixQA planning (§11.4.27 / §11.4.169), fine-grained phased/task/subtask breakdown, enterprise-

> scalability + max-performance planning, OpenDesign UI/UX wireframes where applicable (§11.4.162 / §11.4.190), full CodeGraph integration (§11.4.78 / §11.4.79 / §11.4.80), and fully-synced workable items across the SQLite SSoT + every connected external tracker (§11.4.93 / §11.4.95 / §11.4.148 D5), honestly SKIPPING any absent tracker (§11.4.10). The multi-day research itself is EXECUTED LATER by the standing autonomous loop (§11.4.87 / §11.4.94 / §11.4.97 / §11.4.103 / §11.4.126) when it claims the item, driven to a genuinely COMPLETED-and-wired or explicitly evidence-backed CLOSED terminal state under §11.4.197 — never left un-wired in the backlog; "scheduled" is never reported as "done" (§11.4.6).
>
> The system is UNIVERSAL (every CLI agent reads this block via its context carrier per §11.4.35), extensible (new action = new registry row), decoupled + reusable (§11.4.28), and loads out-of-the-box. Classification: universal (§11.4.17). **Canonical authority:** constitution submodule `Constitution.md` §11.4.140. Non-compliance is a release blocker. No escape hatch — no `--skip-action-prefix`, `--ignore-prefix`, `--no-registry`, `--invent-expansion-OK`, `--single-layer-only` flag.